



Yazılım Mhendisliđine Giriř

- Yazılım mühendisliğine giriş,
- Yazılım mühendisliği ve etik,
- Yazılım mühendisliğinin önemi ve gereği,
- Yazılım geliştirme süreci
- Yazılım sürecinde araştırma, ölçme, planlama ve gereksinim analiz yöntemleri,
- Yazılım mühendisliğinde metodolojileri,
- Yazılım yaşam döngüsü sürecinde metotlar,
- Yazılımda standartlar, kalite teknikleri ve kalite prensipleri,
- Yazılımda proje yönetimi,
- Yazılım projelerinde başarı ve başarısızlık nedenleri,
- Bilgisayar destekli yazılım araçları ve örnek uygulamalar.

- Mesleki etik kuralların öğrenilmesi
- Gelişen teknolojiler ışığında, yazılım mühendisinde bulunması gereken niteliklerin belirlenmesi.
- Yazılım mühendisliği sürecini anlama, uygulama ve proje sürecini yönetebilme becerisi kazandırma.
- Geçerli yazılım mühendisliği standartlarını araştırma, öğrenme ve uygulayabilme.
- Güncel yazılımlar hakkında bilgiler vermek, uygulamaya yönelik çözümler üzerinde durmak.
- Hızla gelişen yazılım ihtiyaçlarına, kabul gören standartlarda güvenli kod geliştirebilme kabiliyeti kazandırma.

Amaç :

Yazılım mühendisliği sürecindeki

- ✓ **isterler,**
- ✓ **analiz,**
- ✓ **tasarım,**
- ✓ **gerçekleme,**
- ✓ **test,**
- ✓ **bakım,**

gibi kavramları öğrenmek, kodlama ilkelerini gözden geçirmek, günümüzün en popüler yazılım projelerinin örneklerini paylaşmak, modüler yapıların incelenmesi, kişisel gelişim ve teknolojik değişime ayak uydurmak.

Ders İçeriği ;

1. (1 .Hft) Yazılım nedir, Yazılım mühendisliği ve eğitimi,
2. (2 .Hft) Yazılım İsteklerin belirlenmesi ve geliştirme süreci,
3. (3 .Hft) Yazılım İsterleri çözümlemesi,
4. (4 .Hft) Veri Akış diyagramı geliştirme, örnek uygulama,
5. (5 .Hft) Yazılım tasarımı,
6. (6 .Hft) Arayüz tasarımı
7. (7 .Hft) Yazılım gerçekleştirimi
8. (8 .Hft) Örnek Uygulamalar
9. (9 .Hft) Yazılım testi ve bakımı
10. (10.Hft) Yazılım kalitesi ve standartları
11. (11.Hft) Proje yönetimi
12. (12.Hft) Örnek Uygulamalar
13. (13.Hft) Başarılı Projelerin Ortak Yönleri
14. (14.Hft) Bilişim Sistemlerinde Değerlendirme ve Etik.

Bu ana başlıklarla verilen ders içeriğinde kod üretme teknikleri, Yazılım Mimarileri, CobiT ve popüler yazılım proje örnekleri gibi örnek çalışmalar gösterilecektir.

- Teorik anlatım
- Uygulama sunuları
- Ödev sunumları
- Rapor

DEĞERLENDİRME SİSTEMİ		
YARIYIL İÇİ ÇALIŞMALARI	SAYISI	KATKI PAYI
Ara Sınav	1	70
Kısa Sınav	2	20
Ödev	1	10
TOPLAM		100
Yılıçının Başarıya Oranı		50
Finalin Başarıya Oranı		50
TOPLAM		100

- Yazılım geliştirme sürecine ait metodolojiyi öğrenmesi ve uygulaması,
- Yazılım projesi geliştirebilmeli ve yönetebilmeli,
- Literatür araştırması ve takibi yapabilmeli,
- Ödev ve rapor hazırlayabilmeli,
- Sunum becerisini geliştirmeli,
- ...

Gerçek yaşamda gereksinim duyulan basit yada karmaşık yazılımların tasarımını, üretimini ve bakımını, zaman ve maliyet kısıtlarını da göz önünde bulundurarak etik ve mühendislik yaklaşımıyla öğrencilere tanıtmak ve aynı zamanda çeşitli bireysel araştırmalar ve grup çalışmalarıyla bu süreçlere yönelik uygulamalar yapmalarına olanak sağlamak.

- Mühendislik mi ?
- Bilim mi ?
- Endüstri mi ?
- Ürün mü?
- Sanat mı ?



- ✓ Değişik ve çeşitli görevler yapma amaçlı tasarlanmış elektronik araçların, birbirleriyle haberleşebilmesini ve uyumunu sağlayarak, görevlerini ya da kullanılabilirliklerini geliştirmeye yarayan makina komutlarıdır.
- ✓ Elektronik cihazların belirli bir işi yapmasını sağlayan programların tümüne verilen isimdir.
- ✓ Var olan bir problemi çözmek amacıyla bilgisayar dili kullanılarak oluşturulmuş anlamlı ifadeler bütünüdür.

- ✓ Bir bilgisayarda donanıma hayat veren ve bilgi işlemde kullanılan programlar, yordamlar, programlama dilleri ve belgelemelerin tümü.

- Bilgisayar programlarının tasarımı, geliştirilmesi, test edilmesi ve bakımı konularını ele alan mühendislik dalıdır.
- Temel mühendislik prensiplerinin bu dalda da uygulanması, önceden tahmin edilebilir ve tekrarlanabilir sonuçların daha çok elde edilmesiyle yazılım mühendisliği gerçek bir mühendislik dalı olma yolunda ilerlemektedir.
- Diğer mühendislik dallarıyla karşılaştırıldığında çok yeni olan bu alanda sürekli yeni yöntemler geliştirilmekte ve konu yavaş yavaş belli bir olgunluğa ulaşmaktadır.
- Gelişmekte olan bu alanda halen ihtiyaç çok büyük olduğundan geleceğin mesleklerinden birisi olarak gösterilmektedir.

- Bilgisayar sistemleri artık günlük hayatın her alanında yoğun ve etkin bir şekilde kullanılmakta olduğundan, bankacılıktan otomotiv sanayisine, sağlık bilgi sistemlerinden şirket yönetimine, telekomünikasyon sistemlerinden hava taşımacılığına, çok geniş alanlarda kullanılan bilgisayar sistemlerinin çok önemli ve kritik bir parçasını oluşturuyor.
- Bilgisayar Yazılım Mühendisliği 1968 yılında NATO tarafından gerçekleştirilen bir konferans esnasında ortaya çıkan yeni bir kavram ve yeni bir mühendislik alanı olup, yazılım sistemlerinin mühendislik prensipleri çerçevesinde tasarımı, üretimi ve işletilmesini hedeflemektedir.

“Yazılım Mühendisliği tüm disiplinlerde uygulamaları olan bir alandır. ”

- Yazılım mühendisliği disiplini 1968 yıllarından bu yana oldukça gelişme kaydetmiş; yazılım geliştirme metodolojileri, programlama paradigmaları, programlama dilleri ve çeşitli araçların geliştirilmesiyle hayli ilerleme katetmiştir.
- Gelişime , IEEE (IEEE Computer Society) ve ACM (Association for Computing Machinery) gibi mesleki kuruluşların önemli etkisi olmuştur.

- Ayrıca bu kuruluşlar son yıllarda, yazılım mühendisliği çekirdek bilgisinin tanımlanması ve bu bilgilerle uyumlu yazılım mühendisliği eğitim programlarının geliştirilmesine yönelik çalışmalar da yapmaktadır.
- Bu bağlamda, diğer mühendislik dallarında olduğu gibi yazılım mühendisliği için de ayrı eğitim programlarının oluşturulması gündeme gelmiştir. Yazılım mühendisliği disiplinin olgunlaşma sürecinde yazılım mühendisliği eğitimi özel bir önem kazanmıştır.

Özel bir önem kazanan bu eğitim programı için özel projelere başlanmıştır. Genelde kısaltılmış adlarıyla karşımıza çıkabilecek olan bu projeler;

SWEBOK(Software Engineering Body of Knowledge) :

Yazılım mühendisliği çekirdek bilgisinin tanımlanması.

SWCEPP(Software Engineering Code of Ethics and Professional Practice) :

Yazılım mühendisliği etiklerinin tanımlanması.

SWEEP (Software Engineering Education Project) :

SWEBOK ile uyumlu olarak örnek bir eğitim programı tanımlanması, olarak sıralanmaktadır.

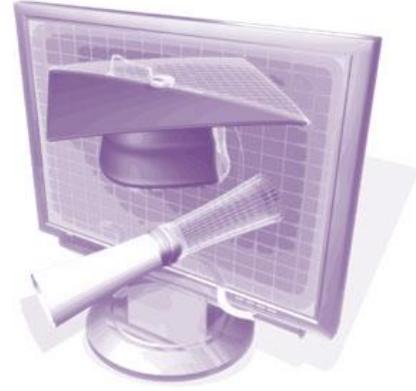
Sonuç ;



Yazılım Mühendisliği eğitiminde teknik bilgi ve beceriler yanında hukuki kavramlar, etik değerler, takım çalışması, proje yönetimi gibi soyut fakat önemli kavramların da kişiye kazandırılması amaçlanmıştır.

- ✓ Sarıdoğan , E., Yazılım Mühendisliği, Papatya Yayınevi,2004.
- ✓ Arifoğlu, A., Doğru, A., Yazılım Mühendisliği, Sas Bilişim Yayınları,2004.
- ✓ Ders Notları.

Yazılım Mühendisliği Eğitimi



Bir yazılım mühendisliği (lisans) mezununun sahip olması gereken yetenekler şunlardır :

1. Yazılım ürünleri geliştirmek için **bir takımın parçası** olarak çalışmak,
2. **Kullanıcı gereksinimlerini** belirlemek ve onları **yazılım gereksinimlerine** çevirmek,
3. Çelişen amaçları düzenlemek, maliyet, zaman, bilgi ve organizasyon kısıtlamaları içinde kabul edilebilir uzlaşmalar bulmak.
4. Bir veya daha çok uygulama alanı için, etik, sosyal, yasal ve ekonomik ilgileri bütünleştiren mühendislik yaklaşımlarını kullanarak uygun çözümler tasarlamak.
5. Yazılım tasarımı, geliştirilmesi, gerçekleştirimi ve doğrulanması için bir temel sağlayan mevcut teorileri, modelleri ve teknikleri anlamak ve uygulayabilmek.
6. Tipik bir yazılım geliştirme ortamında etkin olarak çalışmak, gerekli olduğunda liderlik yapabilmek ve kullanıcılarla iyi iletişim kurabilmek.
7. Yeni modelleri, teknikleri ve teknolojileri öğrenebilmek.

Yazılım Mühendisliği Eğitimi Bilgi Alanları ;

- Temeller
- Profesyonel Uygulama
- Gereksinimler
- Tasarım
- Yazılım oluşturma
- Yazılım sına ve doğrulama
- Yazılım gelişimi
- Yazılım Süreci
- Yazılım Kalitesi
- Yazılım Yönetimi

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Temeller:

Yazılım mühendisliğinin ürettiği ürünlerin niteliklerini anlatan teorik ve bilimsel temellerden, bu ürünleri modellemeyi ve tanımlamayı kolaylaştıran matematiksel temellerden ve öngörülebilir sonuçlar üreten ana ilkelerden oluşur.

Buradaki ana nokta, kaynakları belirlenmiş bir amaca dönüştürmek için mühendislik tasarımı ve mühendislik biliminin uygulanmasıdır.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Profesyonel Uygulama:

Yazılım mühendislerinin, yazılım mühendisliğini profesyonel ve etiğe uygun olarak uygulayabilmeleri için sahip olmaları gereken bilgi, beceri ve davranışlarla ilgilidir.

Profesyonel uygulamalar bilgisi teknik iletişim, psikoloji ve sosyal ve mesleki sorumlulukları içerir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Gereksinimler:

Bir sistemin amacını ve hangi içerikte kullanılacağını tanımlar.

Gereksinimler, kullanıcıların gerçek gereksinimleri ile yazılım ve diğer bilgisayar teknolojileri arasında köprü oluşturur.

Gereksinimlerin belirlenmesi, sistemin fizibilite çalışmasını, kullanıcıların gereksinimlerinin analizi, sistemin ne yapacağını ve ne yapmayacağını kısıtlamalar göz önünde alınarak belirlenmesini ve bu bilginin kullanıcılar tarafından doğrulanmasından oluşur.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Tasarım:

Bir bileşenin veya bir sistemin nasıl gerçekleştirileceğini belirlemek için kullanılan teknikler, stratejiler, gösterimler ve desenlerle ilgilidir.

Tasarım, kaynaklar, performans, güvenilirlik ve güvenlik gibi kısıtlamalar göz önüne alınarak işlevsel gereksinimlere uygun olmalıdır.

Ayrıca, yazılım bileşenleri arasındaki içsel ara yüzler, mimari tasarım, veri tasarımı, kullanıcı ara yüzü tasarımı, tasarım araçları ve tasarımın değerlendirilmesi de bu alanın kapsamındadır.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Oluşturma:

Bu alan, tasarımda belirlenmiş yazılım bileşenlerinin geliştirilmesiyle ilgili bilgileri içermektedir.

Bu kapsamda, bir tasarımın bir gerçekleştirim diline çevrilmesi, bileşen sınamaları ve program belgelemeleri incelenmektedir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Sınama ve Doğrulama:

Elde edilen programın hem belirlenen gereksinimleri sağladığını hem de gerçekleştirimin beklenenlere uygun olduğunu kontrol etmek için statik ve dinamik sınama teknikleri kullanır.

Statik teknikler, yazılımın tüm yaşam döngüsü boyunca elde edilen gösterimlerin analizi ve kontrolüyle ilgilenirken, dinamik teknikler sadece gerçekleştirilmiş sistemi içerir.

Yazılım Gelişimi:

Yazılımın kullanıma verilmesinin öncesindeki ve sonrasındaki aşamalarda etkin bir maliyetle desteklenmesini sağlar.

Bu destek, gelişen sistemi oluşturan versiyonların veya sürümlerin her biri için hazırlık aktivitelerine gerek duyar.

Bu aktiviteler, planlama, ölçüt desteği, regrasyon sınama ve karmaşıklık kontrolünü içermektedir.

Bu aktiviteleri desteklemek için kullanılan teknikler, program anlama, sürüm planlaması, değişiklik tanımlaması, yeniden mühendislik, tersine mühendislik, bakım, sistemin kullanımına son verilmesini içerir.

Yazılım Mühendisliği Eğitimi Bilgi Alanları

Yazılım Süreci:

Yaygın olarak kullanılan yazılım yaşam döngüsü süreç modellerinin tanımlanmasıyla ilgili bilgileri ve kurumsal süreç standartlarını;

yazılım süreçlerinin tanımlanmasını,

gerçekleştirilmesini,

ölçülmesini,

bakımını,

yönetimini,

değiştirilmesi ve iyileştirilmesini ;

ve yazılım geliştirme ve bakımı için gereken teknik ve yönetsel aktiviteleri gerçekleştirmek için tanımlı bir süreç kullanımını kapsamaktadır.

Yazılım Yönetimi:

Tüm yazılım yaşam döngüsü aşamalarının planlanması, düzenlenmesi ve izlenmesiyle ilgili bilgileri içermektedir.

Yazılım geliştirme projelerinin başarısı için, farklı organizasyonel birimlerdeki işlerin koordinasyonu için, yazılım versiyonlarının bakımı için, kaynakların gerekli oldukları zaman var olabilmesi için, projedeki işlerin uygun olarak bölünebilmesi için, iletişimin kolaylaşması için kritik önemdedir.

Yazılım Kalitesi:

Yazılım geliştirmenin ve bakımın tümünü etkileyen ve tümünden etkilenen bir kavramdır.

Hem geliştirilen ürünlerin kalitesini hem de bu ürünleri geliştirmek için kullanılan süreçlerin kalitelerini içerir.

Ürün kalite nitelikleri,

1. kullanılabilirlik,
2. güvenilebilirlik,
3. güvenlik,
4. bakıma uygunluk,
5. esneklik,
6. Etkinlik,
7. performans

gibi kriterleri kapsamaktadır.

Temel Bileşenleri

1. Ekip olarak çalışmak için gerekli bilgiyi sağlayan **Etkin İletişim ve Grup Çalışması Yetenekleri.**
2. Öğrencileri gerçek hayat problemlerine hazırlayan **Bir Uygulama Alanında Deneyim.**
3. Öğrencileri gereksinimlerin değişmesi, proje yönetimi, konfigürasyon yönetimi, araç kullanımı gibi konulara hazırlayan **Bir Ekip Projesi.**
4. Öğrencileri gerçek hayat ortamına hazırlayan **Çalışma Ortamında Deneyim.**
5. Ders notu veya ders gibi belirli bir düzende sağlanmayan bilgiyi aramak, değerlendirmek ve kullanmak için gerekli beceriyi kazandırmayı amaçlayan **Yaşam boyu Öğrenme Araçları.**

6. Yazılım ve donanım ürünleri hakkında temel teknik bilgi ve becerileri sağlayan **Bilgisayar Bilimi Temelleri**. Bunlar, programlama dilleri, modelleme, veritabanları, işletim sistemleri, ağ sistemleri, algoritmalar olarak sayılabilir.
7. Yazılımı ve ilgili belgelemeyi oluşturmak ve bakımı yapmak için gerekli teknik bilgi, yetenek ve araçları sağlayan **Yazılım Mühendisliği Temelleri**. Bunlar arasında, yazılım süreçleri, yaşam döngüsü modelleri, yazılım ölçütleri, mimari ve tasarım yöntemleri bulunmaktadır.
8. Genel sistem ilkelerinin, ekonominin ve mühendislerin görev ve sorumluluklarının anlaşılmasını sağlayan **Mühendislik Uygulamaları ve Etiği**

Yazılım mühendisliği

6 seneye yayılmış bir Eğitim programıdır.

Bu program 2 bölüme ayrılır:

- Temel Yazılımcılık
- Uzman Yazılımcılık

Temel yazılımcılık :

Temel yazılımcılığın süresi 4 Dönemdir.

Bu dört dönem içerisinde yazılımcı adayı bilgisayar yazılımcılığı hakkında temel bilgiler edinir.

- Algoritma geliştirme,
- Sistem analizi,
- Veri tabanı analizi,

Yazılım dilleri hakkında genel bilgiler....

- C ----- C++ ----- Visual C ---- Borland C#
- Assembly (8/16/32/64)
- Sistem analizi
- Veri Tabanı analizi
- Delphi ----- Pascal----Delphi400
- Java --- JavaScript---- Perl ---- Web Tasarım --- CQI --- CGI --- ClientSERVER
- Grafik
- Animasyon
- Projeksiyon --Promasyon --- Prodüksiyon
- . net
- AS400 ----- RPG400
- Ağ yönetimi
- Cracker
- Hacker
- ...

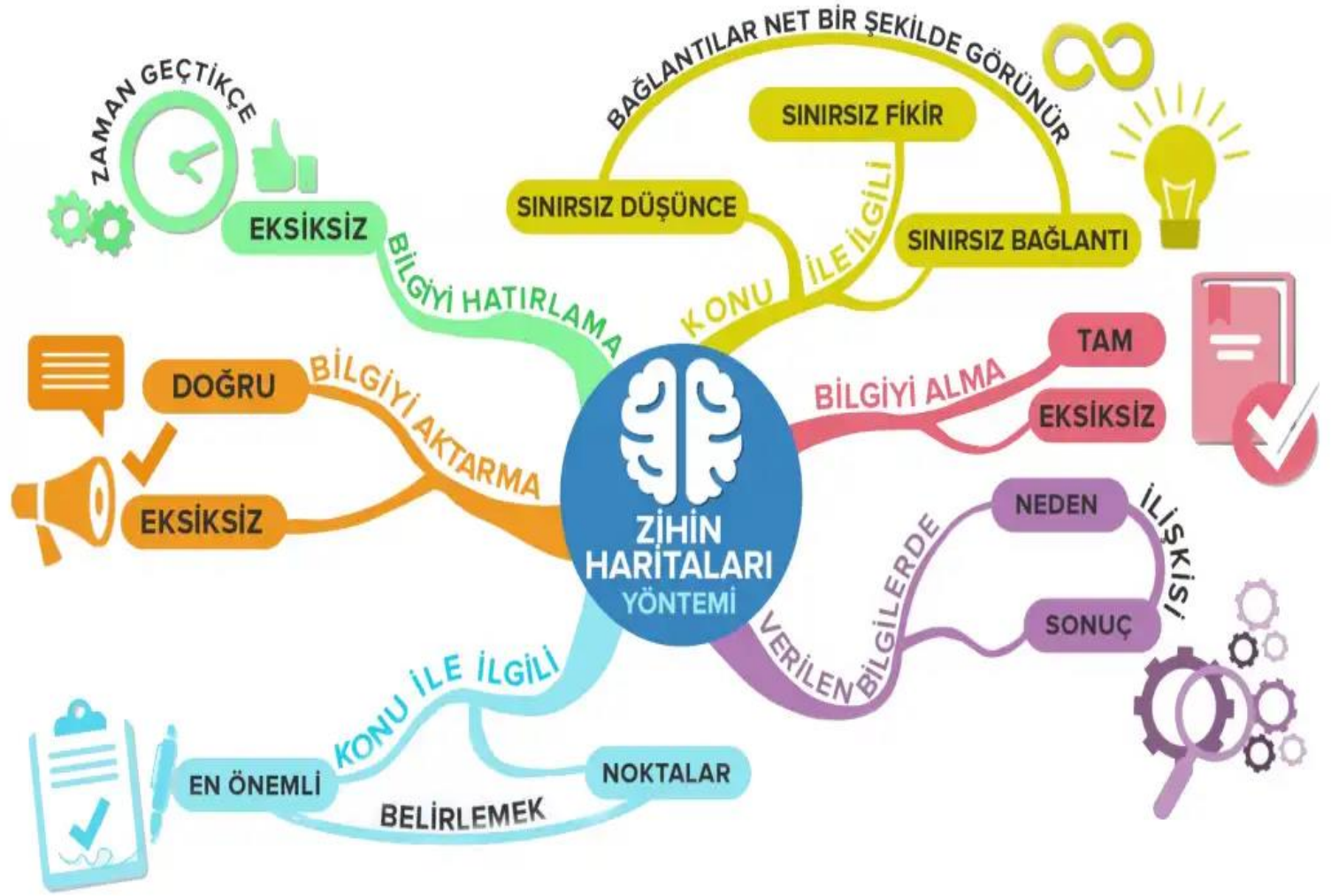
Uzman Yazılımcılık:

Yazılım Mühendisi adayı yukarıda belirtilen konulardan herhangi birini seçer. Daha sonra bu konu hakkında ayrıntılı bilgi edinirler ve uzmanlaşırlar.

“Böylece üretilen işin kalitesi gün geçtikçe artar.”

**Orta büyüklükte bir Yazılım şirketi
en az şu yazılımcıları bulundurmak zorunda kalacak.**

- Sistem Analisti Yazılım Mühendisi
- Hacker Yazılım Mühendisi
- Cracker Yazılım Mühendisi
- Algoritma Yazılım Mühendisi
- Veri Tabanı Analisti Yazılım Mühendisi
- Grafiker Yazılım Mühendisi
- Animasyon Yazılım Mühendisi
- Assembly Yazılım Mühendisi
- Herhangi bir dil kullanan yazılım Mühendisi







Uygulama ...

Kaynak :

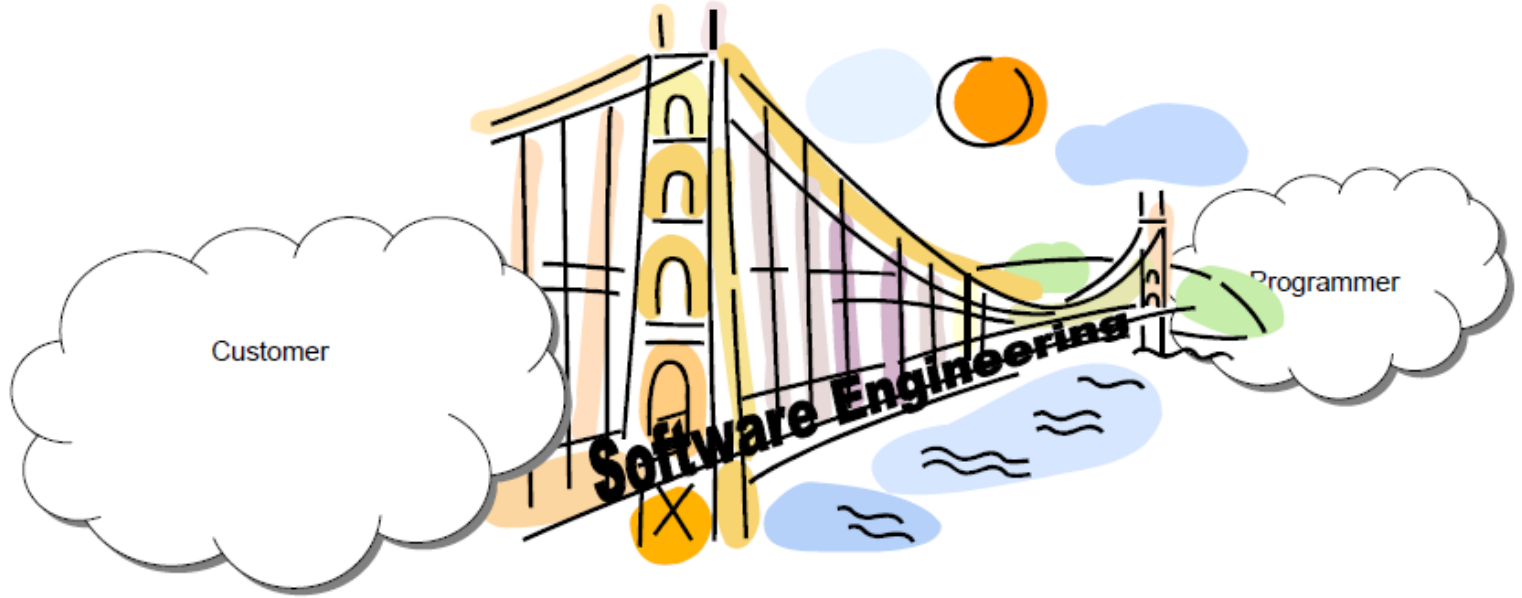
N.Y. Topaloğlu

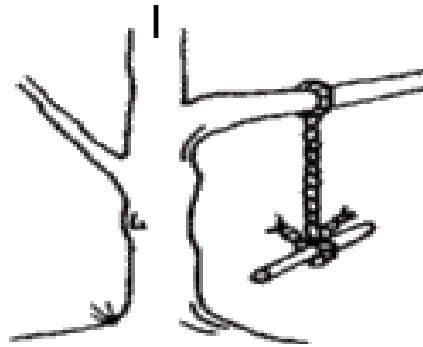
Ege Üniversitesi

Mühendislik Fakültesi

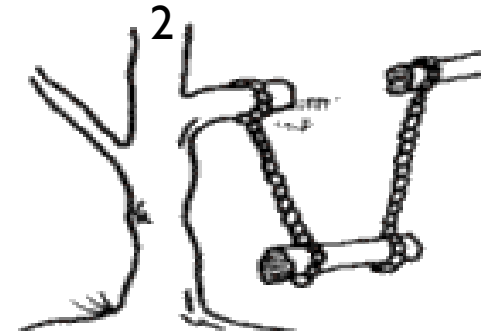
Bilgisayar Mühendisliği Bölümü

Yazılım Mühendisliği

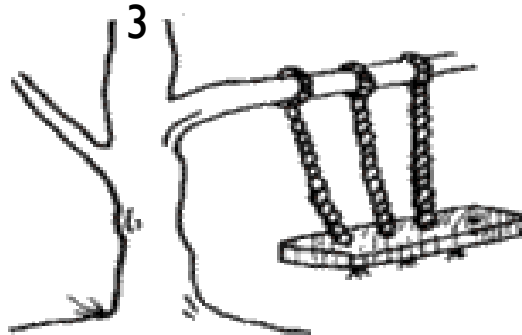




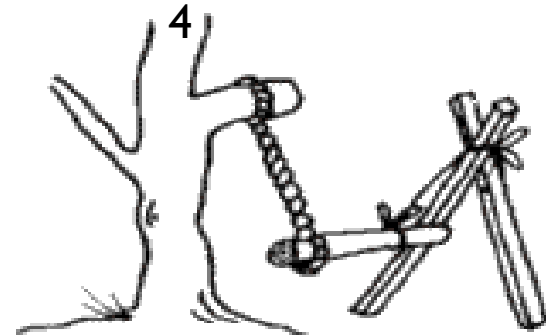
What the user asked for



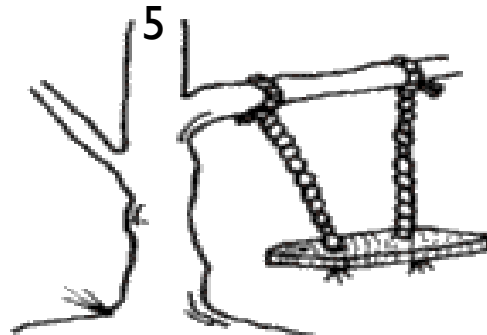
How the analyst saw it



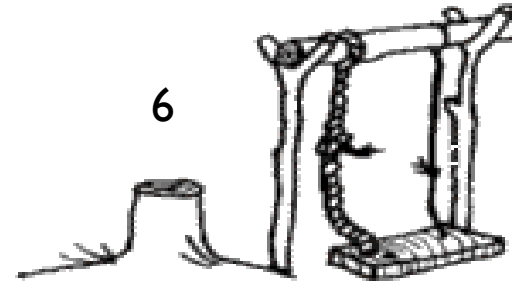
How the system was designed



As the programmer wrote it

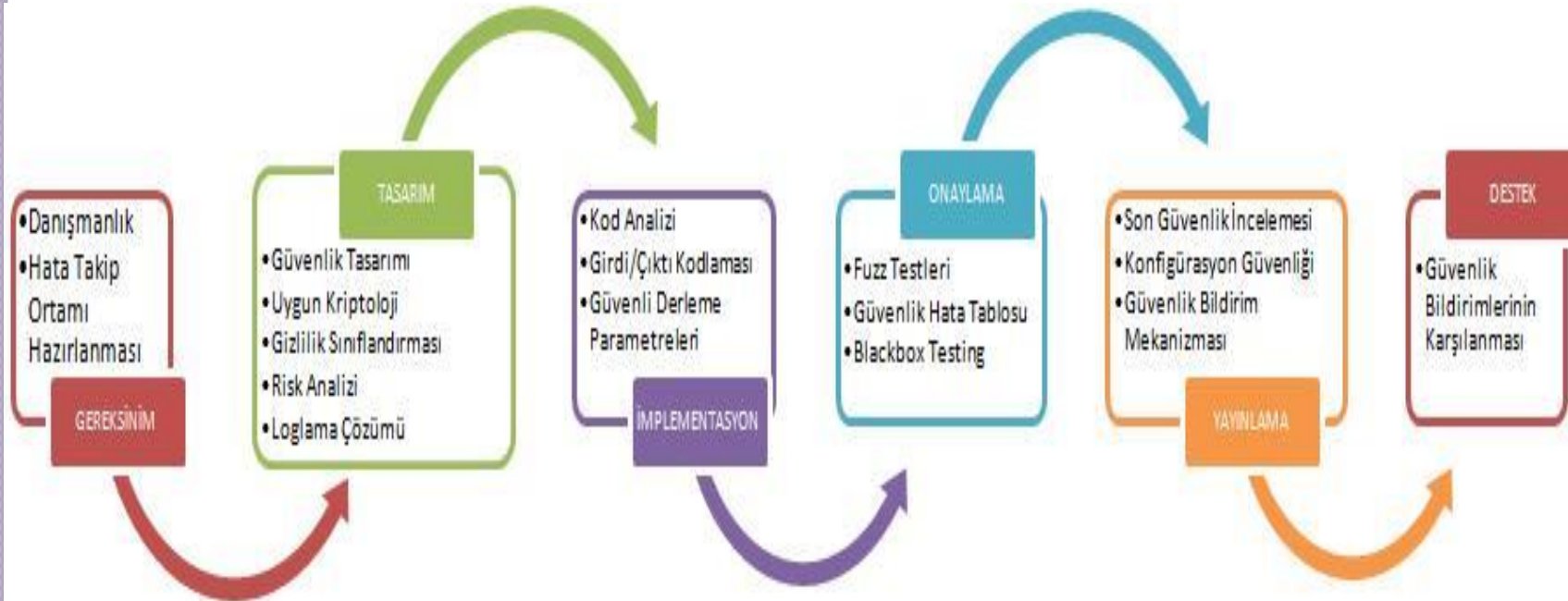


What the user really wanted



How it actually works

Yazılım Yaşam Döngüsü (YYD)

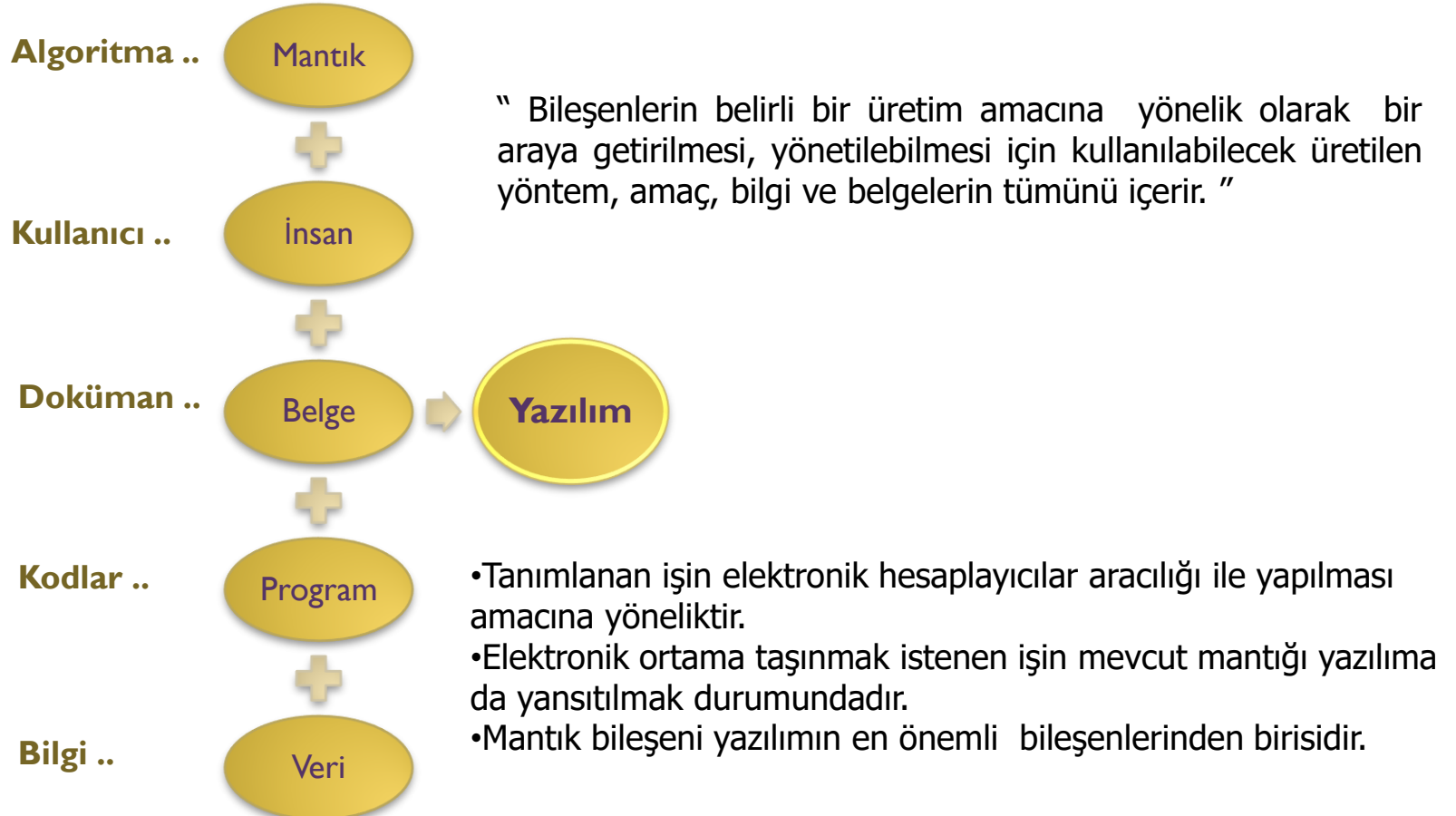


Temel Kavramlar ;

Yazılım: Bir sistemin donanım bileşenleri dışındaki her şey olarak tanımlanabilir.

Yazılım; Bilgisayar program parçası yada programlar grubu olarak an

Yazılımı oluşturan bileşenler



- Her tür yazılım mutlaka bir veri üzerinde çalışma durumundadır.
- Veri dış ortamdan alınabilir yada yazılım içerisinde üretilebilir. 'veri' yi 'bilgi' ye dönüştürme en önemli amaçtır.
- Yazılım üretimi, bir mühendislik disiplini gerektirir.
- Yazılım Yasam Döngüsü (YYD) mühendisler tarafından üretim sırasında kullanılan yasam döngüsünden esinlenerek oluşturulmuştur.
- Yazılım üretimi sırasında, birçok aşamada yapılan ara üretimler, bilgi belge üretimidir.
 - Planlama bilgileri,
 - Çözümleme bilgileri,
 - Tasarım bilgileri,
 - Gerçekleştirim bilgileri

Yazılımın insan bileşeni iki boyutludur.

1. Yazılımı geliştirenler
2. Yazılımı kullananlar

“Çok kişili ekiplerle geliştirilmektedir. Temel nedeni yazılımın yaygınlaşması ve boyutlarının büyümesidir.”

Yazılımın ana çıktısı sonuçta bir bilgisayar programıdır.

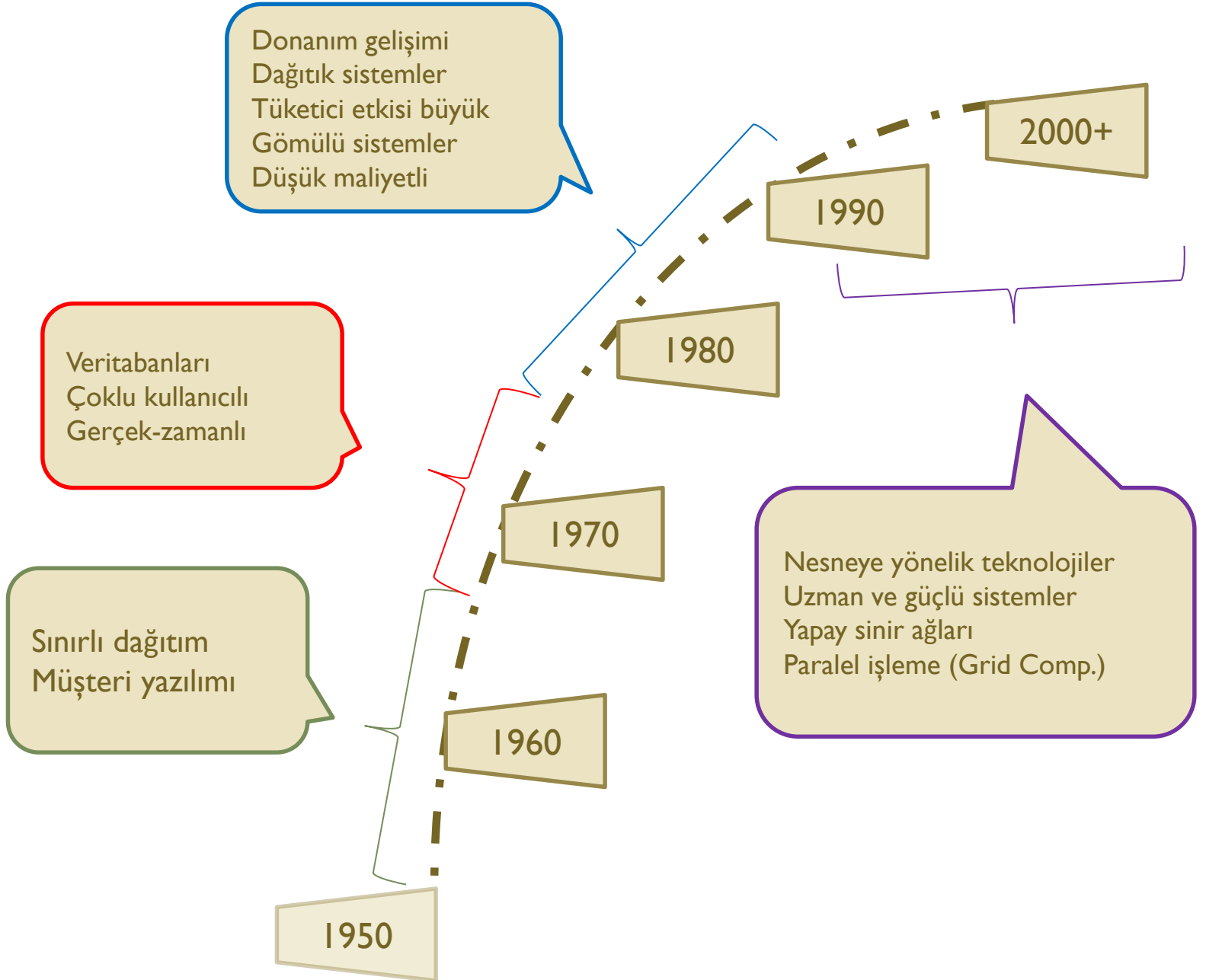
- İşletime alınan programın hemen ardından bakım çalışmaları sürekli olarak gündeme gelir.
- Hiçbir program bütünüyle her olasılık göz önüne alınarak sınınamaz, dolayısıyla hata ihtimali her zaman mevcuttur.
- İşletmeler doğaları gereği dinamik bir yapıya sahiptir, dolayısıyla mevcut sistemin sürekli olarak yeni istek ve gereksinimleri ortaya çıkar.
- Ortaya çıkan talepler ve değişiklikler aynı disiplin içerisinde sisteme eklenmelidir.

“Değişik bilgisayar bilimi teknolojilerinin ve kisilerin bir bilgi yada yazılım sistemi olusturmak amacıyla bir araya getirilmesinde bir bütünleştirici gibi çalışır.”

“Yazılım mühendisi bir programcı değildir, ancak programcının tüm yeteneklerine sahiptir.”

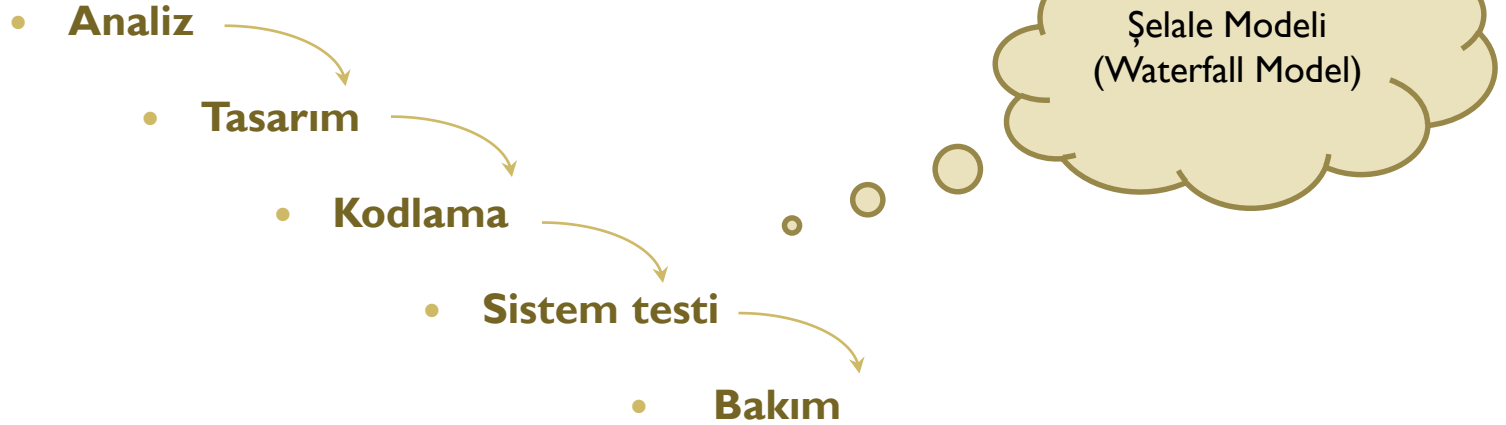
“Yazılım projelerinde temel hedefi, söz konusu üretimin az maliyet, yüksek nitelik ve talebe uygun yapılmasıdır.”

Yazılım Gelişim Süreci :



Yazılım geliştirme adımları

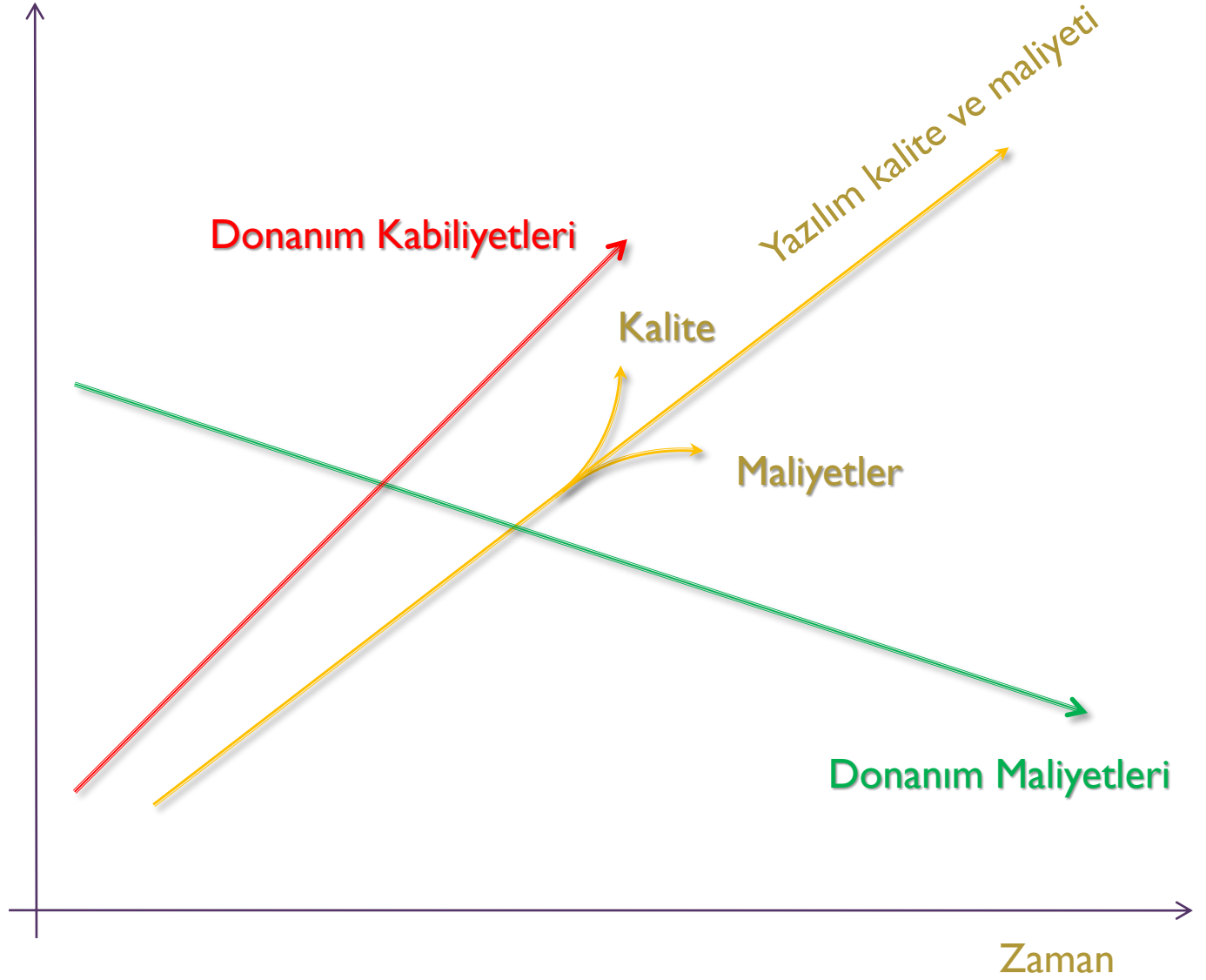
- Gereksinim analizi (requirements analysis)
 - Fonksiyonel ve fonksiyonel-olmayan gereksinimler (functional & non-func. reqs.)
- Tasarım (design)
 - Sistem tasarımı (system design): subsystems
 - Detaylı tasarım (detailed design)
- Kodlama ve birim testi (unit testing)
 - Birleşenlerin ayrı ayrı gerçekleştirilmesi ve birim testi
- Bütünleme testi ve sistem testi
 - (integration & system testing)
- Bakım ve güncelleme (maintenance)
 - Hataların giderilmesi, performans iyileştirme,
 - servislerin geliştirilmesi, değişikliklere uyum, vs.



Yazılım üretiminin, mühendislik yöntemleriyle yapılmasını öngören ve bu yönden yöntem, araç, teknik ve metodolojiler üreten bir disiplindir.

- Yöntemler
- Araçlar
- Teknikler

“Yazılım üretiminde (Yaşam döngüsü) belirtilen aşamaların sistematik olarak izlenmesi ve gerçekleştirilebilmesi yazılım mühendisliği için ön şarttır.”



Problemler !

Büyük çaptaki uygulamalar ne kadar test edilsede, hiçbir zaman %100 hatadan arındırılmaz.

Yazılım geliştirme (çoğunlukla) ;

- Proje temellidir
- Yoğundur (bilgi yoğun)
- Genellikle bütçelerini aşar
- Öngörülen zamandan uzun süren
- Zorluklarla doludur
- Zaman alıcıdır
- Hesapta olmayan masraflar gerektirir
- Sürekli devam eden süreçtir.
- ...

Problemler !

- Değişik yetenekte bir çok personel (raportör, programcı, test sorumlusu, çözümleyici),
- Yeniliğe ve değişime tepki gösteren kullanıcı ve yöneticiler,
- Standart ve yöntem eksiklikleri,
- Yeterince tanımlanmamış, oldukça karmaşık kullanıcı beklentileri,
- Personel sirkülasyonunun fazla olması,
- Yüksek eğitim maliyetleri,
- Dışsal ve içsel kısıtlar (maliyet, zaman, işgücü),
- Verimsiz kaynak kullanımı,
- Mevcut yazılımlardaki yetersizlik ve kalitesizlik,
- Üretim maliyetlerinin yüksek olması,

Hatalar

Yazılım üretiminde Hataların Dağılımı :

- Mantıksal Tasarım %25
- İşlevsel Tasarım %15
- Kodlama %35
- Belgeleme ve Diğer işlevler %25

Yazılım Üretiminde Hata Düzeltme Maliyetleri :

- Çözümleme %1
- Tasarım %5
- Kodlama %15
- Test %30
- Kabul Testi %40
- Uygulama %100

Mühendislik !

Yazılımın mühendislik olarak ifade edilmesi demek ;

Belirli standartlara uyması ve **ölçülebilir olması** gerekir.

- Kalite standartları,
- Kalitenin ölçülebilmesi
- Verimin standartları,
- Verimin ölçülebilmesi

Kalite

- Hata sayısının düşük düzeyde olması
- Kullanıcı isterlerine cevap oluşturabilme (tamamını gerçekleştirebilme)
- Arızalar arası zamanın uzunluğu
- Destek ve gelişme.

Yazılımlar

- Sistem yazılımı
- Gerçek-zamanlı yazılım
- İş yazılımı
- Mühendislik ve bilimsel yazılım
- Gömülü yazılım
- Kişisel bilgisayar
- Yapay zeka yazılımı
- ...

Sınıflama

- İşlevlerine göre
- Zamana dayalı ve uygulama alanlarına göre
- Boyutlarına göre

Örnekler

- Hesaplama (Nümerik Çözümleme)
- süreç temelli (Gömülü sistemler)
- Veri isleme (finans sektörü)
- CAD (Sinyal işleme)
- Kural Temelli (Robotik, Yapay Zeka)

Yazılım Mühendisliğinde İçerik Bilgileri



Bilgisayar Temelleri

- Algoritmalar, veri yapıları, programlama dilleri
- İşletim sistemleri, matematik



Yazılım Mühendisliği

- Mühendislik disiplini, çözümleme, tasarım,
- Gerçekleştirim(kodlama), test, bakım, geliştirme



Yazılım Yönetimi

- Süreç yönetimi, Risk yönetimi, Kalite yönetimi
- Geliştirme yönetimi, Kazanım yönetimi



Yazılım Alanları

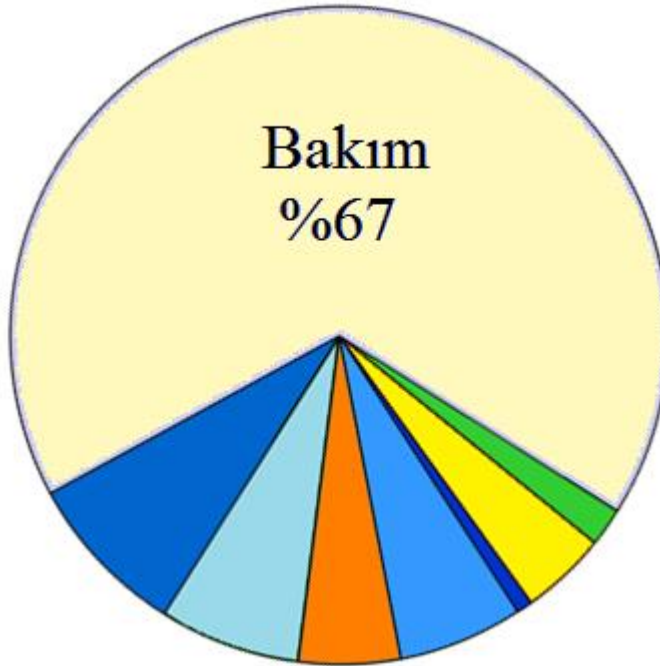
- Veri tabanı, mühendislik, Gömülü sistemler
- Yapay zeka, ...

Süreç bilgi alanları

- Üretimi ve işlemleri içine alan mühendislik disiplini
- Yazılım Mühendisliği Yönetimi
- Yazılım ihtiyaç Analizi
- Yazılım Yönetimi
- Yazılım Tasarımı
- Yazılım Yapılandırılması
- Yazılım Testi
- Yazılım Mühendisliği Altyapısı
- Yazılım Mühendisliği işlemi
- Yazılım Değerlendirme ve Bakımı
- Yazılım Kalite Analizi

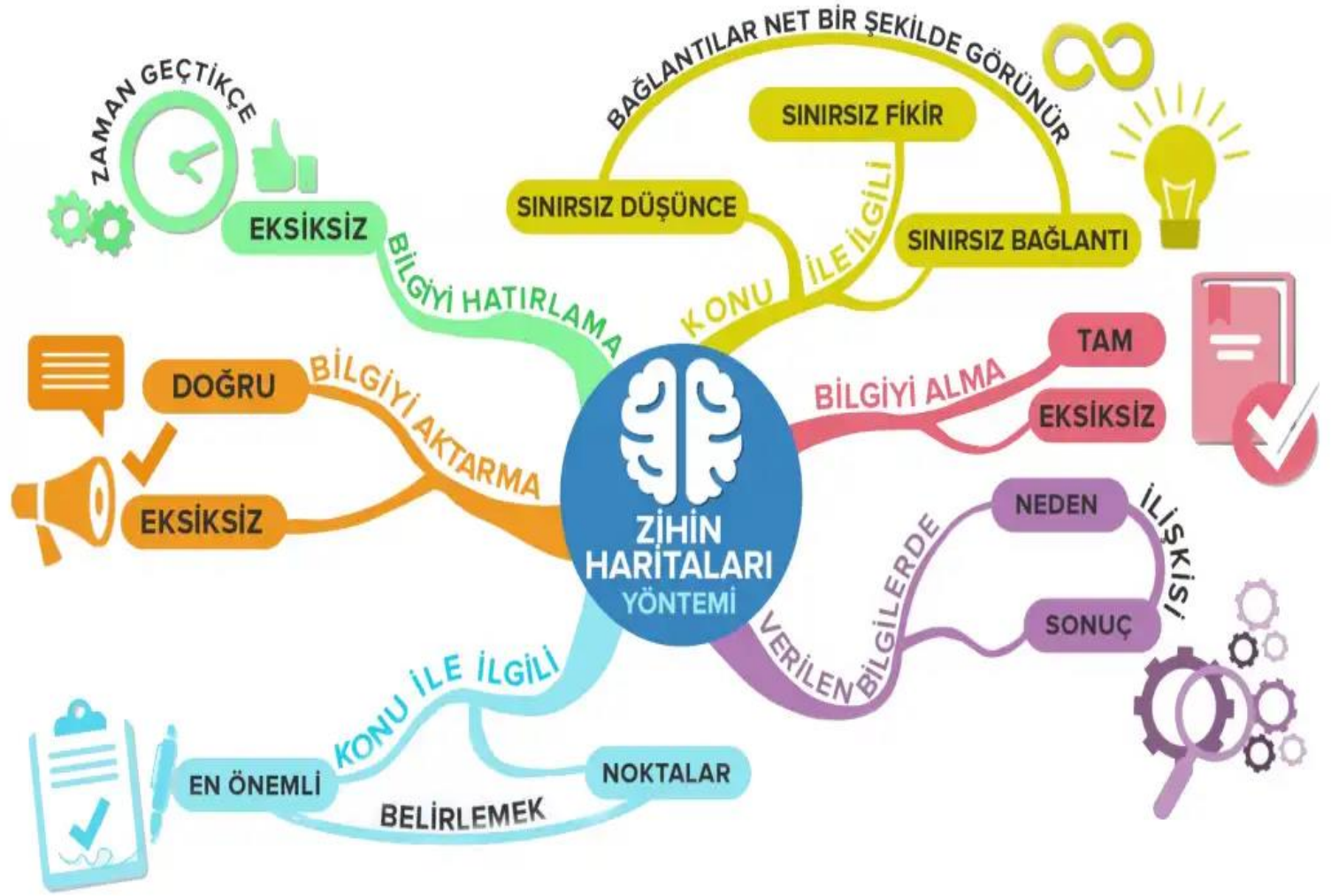
Örnek meslekler ...

- Sistem Analisti
- Yazılım Mimarı
- Yazılım Proje Yöneticisi
- Yazılım Programcısı
- Yazılım Sistem Yöneticisi
- Yazılım Veri Tabanı Yöneticisi
- Yazılım Test elemanı
- Yazılım Ağ Uzmanı
- Yazılım Güvenlik Mühendisi
- Yazılım Konfigürasyon Yöneticisi
- Yazılım Kalite Yöneticisi
- ...



Göreceli maliyetlere göre
yazılım yaşam döngüsü aşamaları.

İhtiyaç belirleme	% 2
Şartname belirleme	% 4
Planlama	% 1
Tasarım oluşturma	% 6
Gerçekleştirim(kodlama)	% 5
Test süreci	% 7
Entegrasyon süreci	% 8
Bakım süreci	% 67







Uygulama ...

Kaynak :

- Roger S. Pressman, Software Engineering – A Practitioner's Approach, 6th Ed., McGraw Hill, International Edition, 2004,
- Prof. Dr. Ş.Sağıroğlu ders notları
- N.Y. Topaloğlu (Makale)
- Wikipedia